One Buffer, Every Scope: A Fixed-Memory Symbol Table for Embedded Compilers, Measured on Real Compiler Traces and Analysed Exactly

Shubhi Singh and Adhyan Jain
Vellore Institute of Technology, Vellore, India

Abstract

Every compiler keeps a scoped symbol table that binds names to declarations and forgets them when a block ends, and compilers increasingly run on microcontrollers, in boot loaders and under fixed memory budgets. There, the table must live in one fixed buffer, refuse a request cleanly when the buffer is full, and bound the work of every call even on hostile source text. Textbook and production designs, such as a hash map with an undo log or one map per scope, rely on a general-purpose allocator, and the analysis of open addressing covers insertions, not the last-in-first-out deletions of scopes. So their memory is unbounded in advance, their behaviour at the limit is undefined, and their probe limits are set with loose bounds. We present STRATA, which keeps an open-addressing index, packed identifier records and an undo stack in one caller-supplied buffer and refuses any call that does not fit without changing a single bit. We prove that popping the newest name is an exact undo, so the table always equals a fresh table of its live names, and that stack order is the only deletion order safe for every table; we compute the refusal probability of a probe-limited table exactly. On symbol-table events recorded from a C compiler building eight open-source projects, STRATA answered all 1 274 180 checked operations correctly, needed a fixed buffer 16.4–17.9% smaller than a conventional single-buffer table at statistically indistinguishable speed on six of eight traces, made 98.7–99.9% fewer allocator calls than its multi-allocation predecessor, and ran on an emulated 64 KiB Cortex-M3. The exact law cuts the probe limit for a failure probability of 10^{-6} at load 3/4 from 455 to 258. Future work is a refusal law for wide scope trees, real hardware and C++ workloads.

Keywords – bounded memory, embedded compilers, linear probing, open addressing, scoped symbol tables, stack discipline

1. Introduction

Compilers, interpreters and virtual machines are moving onto small devices. Run-time systems for Arduino-class microcontrollers execute WebAssembly with live code updates [1], and name resolution has become a subject of its own in mainstream language tooling, where the scheduling of name queries and their safety under parallel checking are characterised with scope graphs [2, 3]. Whatever the language, one data structure sits at the centre of name resolution: a *scoped symbol table* that binds identifiers to declarations, forgets the declarations of a block when the block ends, and is consulted at every use of a name.

Problem and motivation. When the compiler runs on a microcontroller, in a boot loader or under a fixed allocation, three requirements appear. The table must live in *one fixed buffer*, because the allocator is absent or must not be called. It must *refuse* a request that does not fit and stay usable afterwards, because running out of memory is an expected event, not a crash. And the *work* of one call must be bounded, even when the source text comes from someone else: algorithmic denial of service against hash-based structures is a demonstrated attack class, and low-rate adversarial traffic reduced a software packet classifier to about 12% of its capacity [4].

Existing methodologies. The textbook implementation, also used by LLVM, is a hash table from names to their innermost declaration together with an undo log that restores what a scope hid [5, 6, 7]; the other classical design keeps one hash map per scope. Both obtain memory from the allocator one node at a time. On the analytical side, open addressing has a long tradition, from Knuth's analysis of linear probing [8] to exact distributional results [9], and recent years have been productive: iceberg hashing achieves constant time at load $1 - o(1)$ [10], the time/space trade-off of hash tables has been determined for a large class of structures [11], two-way probing makes the worst unsuccessful search $O(\log \log n)$ [12], and quadratic probing has been analysed in a smoothed model [13].

Keyed hash functions such as SipHash are the standard defence against chosen collisions, and reduced-round variants have been cryptanalysed [14]. The combinatorics of the event that every key finds a place are those of parking functions, whose probabilistic properties are now understood in detail [15].

Shortfall. None of these designs gives a table whose memory is fixed in advance, whose behaviour at the limit is defined, and whose cost per call is bounded. Allocator-based tables have none of the three properties; their memory is spread over thousands of small allocations, so even measuring the space a compilation needs is indirect.

The analysis does not cover the case either. It concerns insertions, or deletions in random order; the deletion order of a scoped table is last-in-first-out, and for that order the state of an open-addressing table, its safe deletion orders and its refusal probabilities had not been stated exactly. Probe limits are chosen with Chernoff-type bounds that lose a factor growing with the limit, and whether a probe-limited table refuses an honest name is usually answered by simulation, although tables of a few hundred to a few thousand cells are exactly where asymptotic formulas are inaccurate.

Proposed work. We present STRATA, a scoped symbol table that keeps its open-addressing index, one packed record per visible name and its undo stack in one caller-supplied buffer of 32-bit words, never calls the allocator after it is created, and refuses any call that does not fit without changing a single bit [16]. We measure it against five other tables on symbol-table events recorded from a real C compiler compiling eight open-source projects, on a second compiler, under hash flooding and on an emulated microcontroller, and check every answer against an independent reference model.

We then analyse why the design works and how to configure it. Popping the newest key by clearing its cell is an *exact undo* for every open-addressing scheme that does not move keys, so the table always equals the insertion-only table of its live names, and stack order is the only deletion order safe for every table. On that basis the refusal probability of a probe-limited table is exactly computable, which turns the probe limit and the buffer size into design values instead of guesses. Finally, an interactive tool lets a reader paste a C program, see every name, scope and use, and run the program's symbol-table calls on the real table.

Structure. Section 2 reviews fifteen recent works and their research gaps. Section 3 presents the system: the problem, the architecture, the buffer layout, the exact undo and refusal laws with their equations, the modules, the flow and three algorithms. Section 4 describes the implementation and tools and analyses eight performance metrics with a comparison against related work. Section 5 concludes, Section 6 lists the references and the declarations follow.

2. Related works

Fifteen recent works are reviewed in three groups: hash tables and open addressing (2.1–2.8), scoped name resolution and compilation on constrained devices (2.9–2.12) and adversarial robustness (2.13–2.15). Each summary was written from the abstract of the paper and, for some, its introduction or overview, as recorded with the reference; proofs were not read, and the research gaps are our assessment relative to this work.

2.1 Bender, Kuszmaul and Kuszmaul (2021): linear probing with tombstones [17]

Solution proposed. Shows that tombstones left by deletions create an anti-clustering effect in (ordered) linear probing, so amortized insertion cost at load 1-1/x is x polylog(x) rather than x^2. Also gives 'graveyard hashing', a linear-probing variant with periodic rebuilds that plants tombstones, achieving O(x) expected cost per operation on any operation sequence.

Methodology and tools. Probabilistic analysis of ordered linear probing under insert/delete workloads with rebuild windows of size R. The analysis reduces insertion cost to crossing numbers of intervals, then through a chain of problem transformations to a combinatorial-geometry question about monotone lattice paths (path surplus) with blue/red dots.

The bound is shown tight for hovering workloads in the classical setting ($\tilde{O}(x^{1.5})$ with an $\Omega(x^{1.5} \lg \lg x)$ lower bound); a larger rebuild window R=n/polylog(x) gives $\tilde{O}(x)$. Graveyard hashing is analysed separately (the paper states it needs 5-independent hashing).

Result and effectiveness. Amortized expected insertion cost x polylog(x) at load 1-1/x with queries O(x); graveyard hashing gives O(x) per operation and 1+o(1) block transfers when x=o(B), versus the $B=\Omega(x^2)$ needed earlier.

Research gap. (Our assessment) Analyses expected amortized time under random deletions with tombstones and rebuilds; it does not study LIFO (stack-ordered) deletions with exact undo, nor exact refusal probabilities for a hard probe limit.

2.2 Braverman and Kuszmaul (2024): tight analyses of linear probing [18]

Solution proposed. Settles the amortized insertion cost of linear probing with tombstones at load 1-1/x at $\Theta(x \log^{1.5} x)$, and extends the bounds from ordered linear probing to the classical unordered variant used in practice.

Methodology and tools. Reduces the problem to the path surplus problem (maximum over monotone grid paths of blue minus red Poisson dots) and gives a new analysis showing expected maximum surplus $\Theta(m \log^{0.75} m)$. A tighter reduction from path surplus back to ordered linear probing then yields matching bounds with an optimally chosen rebuild window.

Result and effectiveness. $\Theta(x \log^{1.5} x)$ worst-case amortized insertion/deletion time with $O(x)$ queries; path surplus $\Theta(m \log^{0.75} m)$.

Research gap. (Our assessment) Asymptotic amortized cost under random (not stack-ordered) deletes with rebuilds; no exact probabilities of refusal under a fixed probe limit.

2.3 Farach-Colton, Krapivin and Kuszmaul (2024): elastic and funnel hashing [19]

Solution proposed. Gives 'elastic hashing', a non-greedy open-addressed table without reordering with O(1) amortized and $O(\log \delta^{-1})$ worst-case expected probe complexity, and 'funnel hashing', a greedy scheme with $O(\log^2 \delta^{-1})$ worst-case expected probes. This disproves Yao's 1985 conjecture that uniform probing is optimal among greedy schemes.

Methodology and tools. Constructions partition the array into geometrically organized sub-arrays (levels) so that insertions can probe far down the probe sequence before settling, bypassing the coupon-collector bottleneck. Matching lower bounds are proved: $\Omega(\log \delta^{-1})$ for any non-reordering table, $\Omega(\log^2 \delta^{-1})$ for greedy schemes, and $\Omega(\log^2 \delta^{-1} + \log \log n)$ w.h.p. (details of proofs not read).

Result and effectiveness. Insertion-only open addressing without reordering is solved up to constants: amortized O(1), worst-case expected $O(\log \delta^{-1})$; funnel hashing beats uniform probing's $\Theta(\delta^{-1})$ for greedy schemes, all with matching lower bounds.

Research gap. (Our assessment) Insertion-only model and δ-fraction slack; no deletions, no stack discipline, no hard probe cap with refusals, and the structures are far from the plain linear probing used in compiler symbol tables.

2.4 Krapivin, Kuszmaul and Wang (2026): interlinear probing [20]

Solution proposed. Introduces interlinear probing, a non-greedy insertion rule for linear probing (an insertion may skip a free slot to leave it for later) with no reordering. Given x in advance, worst-case

expected insertion and positive-query time is $O(x \log x)$ at load $1-1/x$, versus $\Theta(x^2)$ for greedy linear probing.

Methodology and tools. Upper bounds: a stable non-greedy variant of linear probing, extended with a new stopping rule so negative queries cost $O(x \log^2 x)$, and a further extension giving $O(x)$ amortized insertion. Lower bound: a multiscale potential argument over power-of-two block sizes compares imbalance from random home locations with the final-placement imbalance, showing $\Omega(x \sqrt{(\log x)})$ for any stable strategy.

Upper bounds assume $x \leq n^{0.49}$ and x known in advance.

Result and effectiveness. Optimal worst-case expected insertion time among stable linear-probing strategies is $x (\log x)^{\Theta(1)}$: $O(x \log x)$ upper bound versus $\Omega(x \sqrt{(\log x)})$ lower bound.

Research gap. (Our assessment) Insertion-only (and the 'no reordering' stable model) with expected time; no deletions, no hard probe cap with refusal probabilities, and no stack-ordered undo.

2.5 Alharbi, Banderier and Bouillaguet (2026): bounded linear probing [21]

Solution proposed. Introduces bounded linear probing: an insertion probes at most p consecutive cells and is discarded (counted as failed) if all are occupied. It interpolates between the coupon collector ($p=1$) and classical linear probing ($p \geq n$). Derives the expected number of insertion attempts needed to fill the table.

Methodology and tools. Analytic combinatorics: tables are grouped by circular symmetry (as in Knuth) into full and almost-full tables with a 'trash bin' of failed insertions. A non-linear PDE for the bivariate generating functions (Theorem 1 and 2) is derived using a full algebra of operators, then solved in closed form (Theorems 3 and 4).

Asymptotics of the expected filling time are obtained from the closed forms (Theorem 5), and compared to experiments. Motivated by long-message second-preimage attacks on Merkle-Damgard hash functions.

Result and effectiveness. Closed-form generating functions for full and almost-full tables for any p, and the asymptotic expected number of insertion trials to fill a table, which decreases by almost a factor p as p grows.

Research gap. (Our assessment) The closest prior work on probe-limited linear probing, but for insertions in which a failed key is discarded and redrawn until the table is full; it has no deletions, no stack discipline, and no refuse-and-stop event with a refusal probability at a fixed load.

2.6 Bender, Kuszmaul and Zhou (2025): non-oblivious open addressing [22]

Solution proposed. Constructs a non-oblivious open-addressed hash table ('partner hashing') with $O(1)$-time operations even when the table is completely full, including a dynamically resized version

using exactly as many slots as elements. Time bounds hold with high probability using only O(1) O(1)-wise independent hash functions.

Methodology and tools. The main technique encodes metadata in the relative ordering of the stored elements, simulating a $\Theta(n \log n)$-bit RAM with $O(\log n)$-bit words inside the permutation of elements, with O(1) access and efficient updates. The intro presents a warm-up structure first and then refinements for resizing and high-probability bounds (read at intro level only; proofs not read).

Result and effectiveness. Constant-time insert/delete/query at load factor 1 (full table), with high probability; this bypasses the $\Omega(\log \log \varepsilon^{-1})$ lower bound that holds for oblivious open addressing.

Research gap. (Our assessment) Gives up the simple structure of classical open addressing (probe sequence plus naive layout) by encoding data in element order; says nothing about plain linear probing with a probe limit, stack-ordered deletes, or exact refusal probabilities.

2.7 Bender et al. (2023): tiny pointers [23]

Solution proposed. Introduces tiny pointers: o(log n)-bit replacements for log n-bit pointers at constant-factor time overhead, with optimal fixed-size and variable-size constructions. Applies them to data retrieval, succinct dynamic BSTs, stable dictionaries, variable-size-key dictionaries and the stash problem.

Methodology and tools. Based on the abstract only: the constructions revisit classic balls-and-bins problems, and the tiny-pointer theory gives optimal sizes $\Theta(\log \log \log n + \log k)$ bits (fixed) and $\Theta(\log k)$ expected bits (variable) at load 1-1/k. Technique details were not read beyond the abstract.

Result and effectiveness. Optimal tiny-pointer sizes at load factor 1-1/k, and space-efficient solutions to five classic data-structure problems 'for free'.

Research gap. (Our assessment) Space-efficiency theory for pointer replacement; not about probe-limited linear probing, refusals, or LIFO deletion. Cite as background on space-time tradeoffs in hashing.

2.8 Aguilar-Fraga et al. (2024): ℓ-interval parking functions [24]

Solution proposed. Generalizes interval parking functions to n cars and $m \geq n$ spots (interval rational parking functions) and counts them, then studies the subset where each car parks at most l spots from its preference (l-interval).

Methodology and tools. Enumerative combinatorics: a counting formula for interval rational parking functions, recursive formulas for l-interval ones, and nondecreasing l-interval counts via the outcome map; for l=1 a bijection with barred preferential arrangements. Beyond the abstract, details were not read.

Result and effectiveness. Enumeration formulas (closed and recursive), a Fibonacci-type count for the intersection with Fubini rankings, and the l=1 bijection with barred preferential arrangements (recovering the unit-interval result of Hadaway and Harris).

Research gap. (Our assessment) The l-interval condition is a probe limit in parking-function language, but the paper counts configurations; it has no hashing interpretation, no deletion/undo, and no probability law for refusals with random keys.

2.9 van Antwerpen et al. (2018): scopes as types [25]

Solution proposed. Treats scopes as types so scope graphs can model the internal structure of non-simple types (structural records, generic classes). Extends scope graphs with scoped relations and queries, and introduces Statix, a meta-language for static semantics based on scope graphs plus constraints.

Methodology and tools. Abstract only (full text not read). The approach extends scope graphs with scoped relations and generalized queries and is realised in the Statix DSL, which specifies static semantics via scope graphs and constraints.

Evaluation, per the abstract, is by case studies: simply-typed lambda calculus with records, System F, and Featherweight Generic Java.

Result and effectiveness. Abstract reports only that the approach and Statix design were evaluated in the three case studies; no quantitative figures are stated in the abstract.

Research gap. Our assessment: Scope graphs are a declarative, unbounded-memory model of name binding and resolution; it does not address a fixed byte buffer, LIFO push/pop of a hash table, probe limits, or the probability that an insertion is refused.

2.10 Zwaan et al. (2022): incremental type checking from scope graphs [26]

Solution proposed. Derives incremental type checkers automatically from Statix specifications: name-resolution queries in scope graphs define dependencies between compilation units, and a query-confirmation algorithm finds queries whose answers changed after an edit so only those units are re-analysed.

Methodology and tools. Abstract only. Technique: dependency derivation from scope-graph queries, a query-confirmation algorithm, splitting checking into context-free and context-sensitive parts, and a generic mechanism for mutual dependencies.

Evaluation compares against non-incremental parallel execution on synthetic benchmarks and real-world projects.

Result and effectiveness. Abstract reports speedups up to 147x on synthetic benchmarks and up to 21x on real-world projects over non-incremental parallel execution, with initial overhead below 10%.

Research gap. Our assessment: Incrementality is about re-checking after edits with unbounded memory; it does not cover bounded-memory tables, deterministic undo on scope exit, or refusal-probability guarantees.

2.11 Ball, de Halleux and Moskal (2019): Static TypeScript [27]

Solution proposed. Static TypeScript (STS), a TypeScript subset with a compiler/linker toolchain written in TypeScript running in the browser. Programs compile to ARM Thumb machine code linked against a precompiled C++ runtime, aimed at Cortex-M devices with 16-256 kB RAM (e.g. micro:bit, 16 kB).

Methodology and tools. Read in the draft: description of the STS language subset (static layout of classes, nominal-style class compilation), the compilation scheme, and the MakeCode deployment context.

Evaluation compares STS (native Thumb and a VM) with C (gcc), Duktape, IoT.js, MicroPython, CPython and Node.js on STM32F401RE (96 kB RAM), ATSAMD51G19 (192 kB RAM) and a Raspberry Pi Zero (reference only), using small benchmarks including binary trees, richards, fann and nbody.

Result and effectiveness. Draft states STS is over an order of magnitude faster than embedded interpreters on property-heavy benchmarks, less than 2x slower than Node.js there, interpreters run out of memory sooner, and STS runs on devices with as little as 16 kB RAM.

Research gap. Our assessment: Shows full compilation (including scoping and typing) targeting tiny devices but runs the compiler in a browser on the host; it does not give a fixed-buffer on-device symbol table or probabilistic analysis of its capacity limits.

2.12 Mochizuki and Yamazaki (2025): BlueScript [28]

Solution proposed. Proposes a 'disaggregated' VM that offloads as many VM components as possible (incl. incremental and dynamic compilers) to a host, using a 'shadow machine' on the host that mirrors microcontroller execution state to cut communication overhead; instantiated as the BlueScript VM.

Methodology and tools. Abstract only. Design and implementation of the BlueScript VM with host-side offloaded components and a shadow-machine data structure.

Experiments compare against MicroPython and Espruino (abstract level; setup details not read).

Result and effectiveness. Abstract reports offloading does not seriously compromise expected benefits; the offloaded incremental compiler gives faster execution than MicroPython and Espruino with interactivity comparable to MicroPython, and the offloaded dynamic compiler improves VM performance.

Research gap. Our assessment: Argues for moving memory-hungry compiler state off the device; leaves open the case where scoped symbol tables must live on the device itself in one fixed buffer.

2.13 Naor and Yogev (2019): Bloom filters in adversarial environments [29]

Solution proposed. Defines an adversarial model in which an adversary picks inputs and queries adaptively from earlier answers, and relates secure Bloom filters to cryptography. Gives a construction against query-bounded unbounded adversaries.

Methodology and tools. Abstract only. Cryptographic reductions and information-theoretic constructions for computationally bounded and query-bounded unbounded adversaries.

Result and effectiveness. Per abstract: non-trivial Bloom filters against efficient adversaries exist iff one-way functions exist; against t queries a filter with $O(n \log(1/\varepsilon) + t)$ bits exists, versus $n \log(1/\varepsilon)$ for non-adaptive adversaries.

Research gap. Our assessment: sets the adaptive-adversary framework we can borrow, but treats approximate membership, not open-addressing tables; no exact finite-size refusal law and no deletion model.

2.14 Clayton, Patton and Shrimpton (2019): probabilistic data structures in adversarial environments [30]

Solution proposed. Gives a provable-security treatment of probabilistic data structures when data and queries may depend on internal randomness. Shows that salts or secret keys yield provable security with little overhead.

Methodology and tools. Abstract only. A syntax covering many structures and security notions that support error bounds under powerful attacks; focuses on Bloom filters, also counting Bloom filters and count-min sketch.

Result and effectiveness. Per abstract: findings for the traditional (unkeyed) structures are largely negative; simple keyed or salted variants give provable security at little overhead.

Research gap. Our assessment: close in spirit to our key-lifetime question, but for approximate-membership/sketch structures; does not give exact probe-limit refusal probabilities or handle stack-ordered undo.

2.15 Blair et al. (2020): HotFuzz [31]

Solution proposed. HotFuzz is a framework that finds algorithmic-complexity (AC) vulnerabilities in Java libraries by micro-fuzzing individual methods. A genetic algorithm evolves Java objects to trigger worst-case performance. Confirmed cases are replayed as standalone Java programs.

Methodology and tools. Abstract only. Micro-fuzzing with a genetic algorithm over Java objects; seed inputs derived by Small Recursive Instantiation (SRI); candidate witnesses are synthesized into Java programs and monitored for high CPU utilization. Evaluated on the JRE, the 100 most popular Maven libraries and DARPA STAC challenges, comparing SRI seeds against empty-value seeds.

Result and effectiveness. Per abstract: verified known AC vulnerabilities, found previously unknown ones reported to vendors (confirmed by IBM and Oracle), and SRI seeds outperformed empty seeds. Abstract gives no counts.

Research gap. Our assessment: finds implementation-level worst-case inputs but gives no exact probability law; it does not treat keyed hashing, probe limits or stack-ordered deletion in a hash table.

3. Proposed work/system

3.1 Problem formulation

A scoped symbol table receives a sequence of four calls from the compiler's front end: `declare(x, v)` binds name *x* to value and type *v* in the current scope, `lookup(x)` returns the innermost visible binding of *x*, `enter_scope()` opens a block and `exit_scope()` closes it, after which every binding made in the block is gone and every binding it hid is visible again. A redeclaration in the same scope is reported. The deletion order is therefore fixed by the program: the bindings of a scope disappear together and newest first.

We require (R1) that all state lives in one buffer of *C* bytes supplied by the caller, with no allocator call after creation; (R2) that a call which does not fit is refused with a status code and leaves the table bit-for-bit unchanged; (R3) that, in bounded mode, every call examines at most *L* index slots whatever the names are; and (R4) that every answer equals that of a reference model of scoped binding. The questions are how small *C* can be on real programs, what the design costs in time against allocator-based and allocator-free alternatives, and how *L* and *C* should be chosen so that an honest program is refused with a stated, small probability.

3.2 System architecture

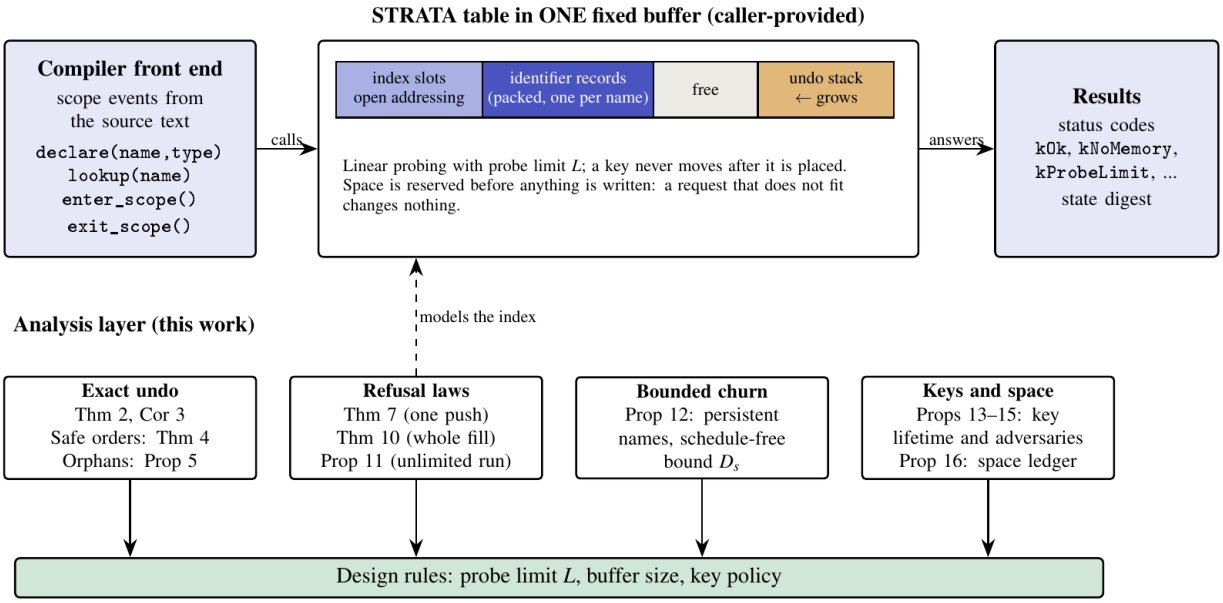


Figure 1. Architecture. The compiler front end issues four calls to a STRATA table that keeps its open-addressing index, packed identifier records and undo stack in one fixed buffer. The analysis layer models the index and produces design rules for the probe limit, the buffer size and the key policy.

Figure 1 shows the system. The compiler front end issues the four calls. STRATA keeps the index, one packed record per visible name and the undo stack of the open scopes in one caller-provided buffer. A declaration that hides an outer name saves the old binding on the undo stack, and leaving a scope pops the entries in reverse order. Space for the record and the undo entry is reserved before anything is written, so a request that does not fit returns `kNoMemory` and leaves the table unchanged; in bounded mode a request whose probe sequence meets L occupied slots returns `kProbeLimit`, also without change.

Below the table, the analysis layer mirrors the requirements: exact undo and safe deletion orders explain why scope exit needs no tombstones (R2, R4), the refusal laws give the probability of `kProbeLimit` (R3), the key-lifetime results say when the laws hold against chosen names, and the space ledger explains the buffer size (R1). The dashed arrow marks the modelling step: the index of the table is a ring of cells, and everything proved about the ring is a statement about the index. Around both sits the measurement harness of Section 4, which replays recorded compiler traces on STRATA and five other tables.

3.3 Detailed methodology

3.3.1 One buffer, four regions

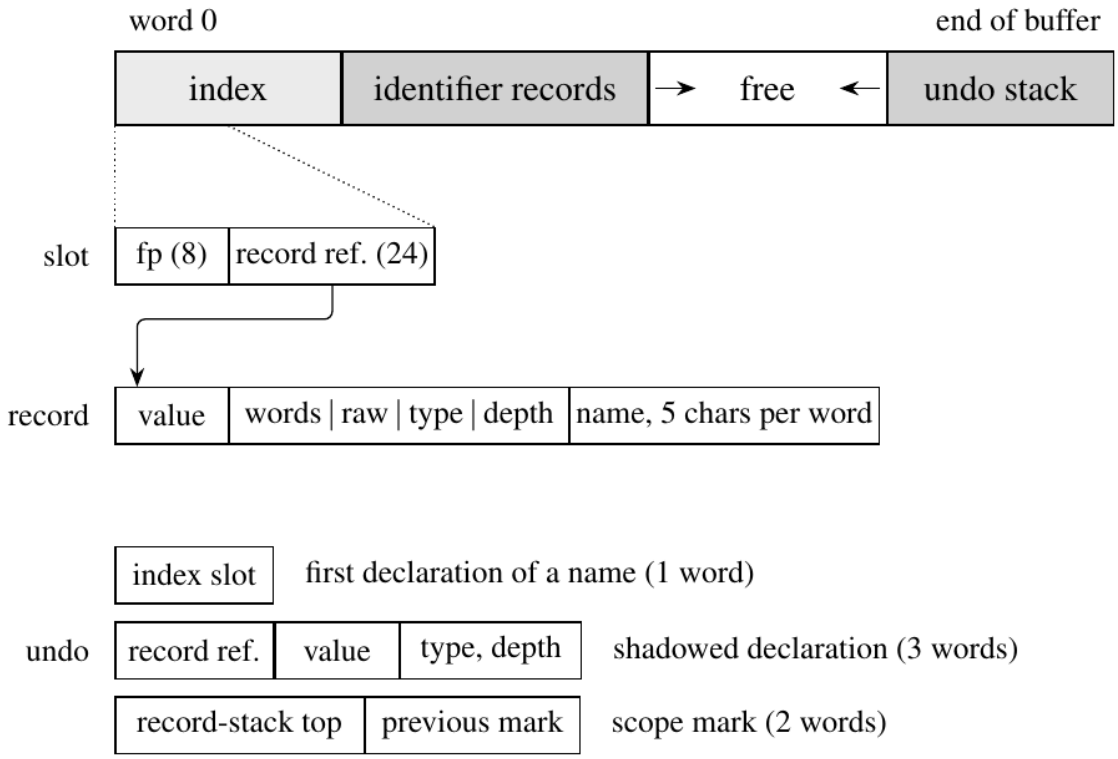


Figure 2. Layout of the buffer and of the three kinds of undo entries. The index and the identifier records grow from the start of the buffer, the undo stack from its end; the free space between them is shared.

All state lives in one array of 32-bit words (Figure 2). The *index* occupies the first m words, m a power of two; a slot is zero when empty and otherwise holds an 8-bit fingerprint and a 24-bit reference to an identifier record. Collisions are resolved by linear probing, and the index is kept at most three quarters full. The *identifier records* follow the index and grow upward, one per distinct visible name; a record holds the value of the name's innermost declaration, a metadata word and the name, packed five characters to a word when it consists of letters, digits and underscores. A lookup therefore ends in the record it finds, whatever the depth of shadowing. The *undo stack* grows downward from the end of the buffer: a new name pushes one word (its index slot), a declaration that hides an outer one pushes three (the record reference and the overwritten value, type and depth), and entering a scope pushes a two-word mark. With n distinct visible names, d live declarations and s open scopes, the buffer is used up to

$$M = m + 1 + \sum_{x} (2 + w(x)) + n + 3\,(d - n) + 2\,s \tag{1}$$

words, where $w(x)$ is the number of name words. A declaration is accepted only if the words it needs fit between the two growing ends:

$$\text{accept} \;\Leftrightarrow\; M + \Delta M \le \frac{C}{4} \tag{2}$$

with $\Delta M = 3 + w(x)$ words for a new name and 3 words for a declaration that hides an outer one. Otherwise the call returns `kNoMemory` before writing anything (all-or-nothing refusal). The conventional allocator-free table used as the fair baseline stores, per declaration, a chain link, the hash, the value, a length-and-depth word and the unpacked name; the difference between the two layouts is quantified exactly by the space ledger of Metric 4.

3.3.2 Exact undo and safe deletion orders

Write $\mathrm{Ins}(a_1,\ldots,a_n)$ for the table obtained by inserting keys in this order into an empty table. **Theorem 2 (undo).** For every open-addressing rule that never moves a key and chooses a cell from the occupied set and the key's own probe sequence, after any sequence of pushes and pops the table at time t is the insertion-only table of the live keys in push order:

$$T_t = \mathrm{Ins}(a_1, \ldots, a_{n_t}) \tag{3}$$

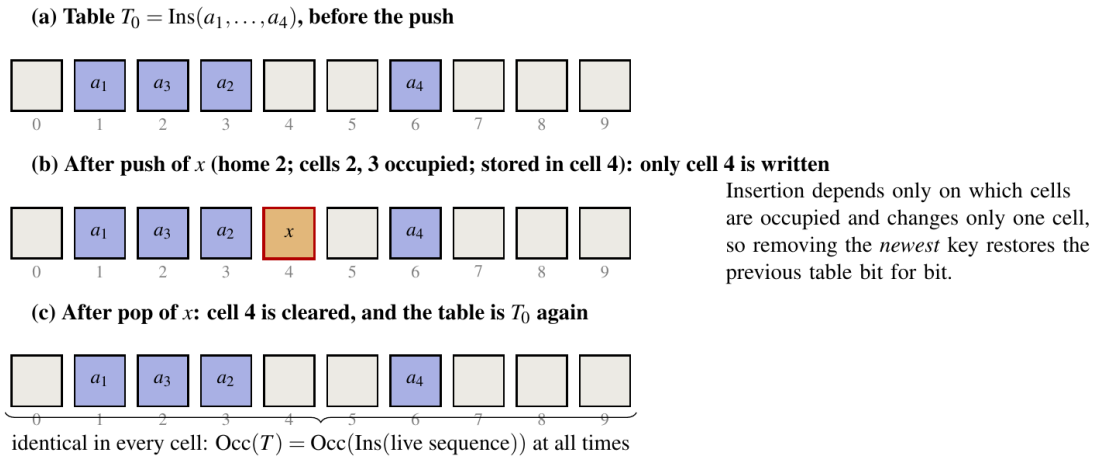


Figure 3. Exact undo. Pushing x writes one empty cell; popping x clears it, and the table is identical in every cell to the table before the push.

The proof is by induction: an insertion changes exactly one empty cell, chosen from the occupied set alone, so clearing that cell restores the previous table (Figure 3). This is why STRATA's scope exit needs no tombstones and no rehashing: the newest key lies on no other key's probe path. A key y *depends on* x, written $y \lhd x$, if the cell of x lies strictly before the cell of y on the probe path of y:

$$y \lhd x \;\Leftrightarrow\; c(x) \in \{h(y), h(y)+1, \ldots, c(y)-1\} \tag{4}$$

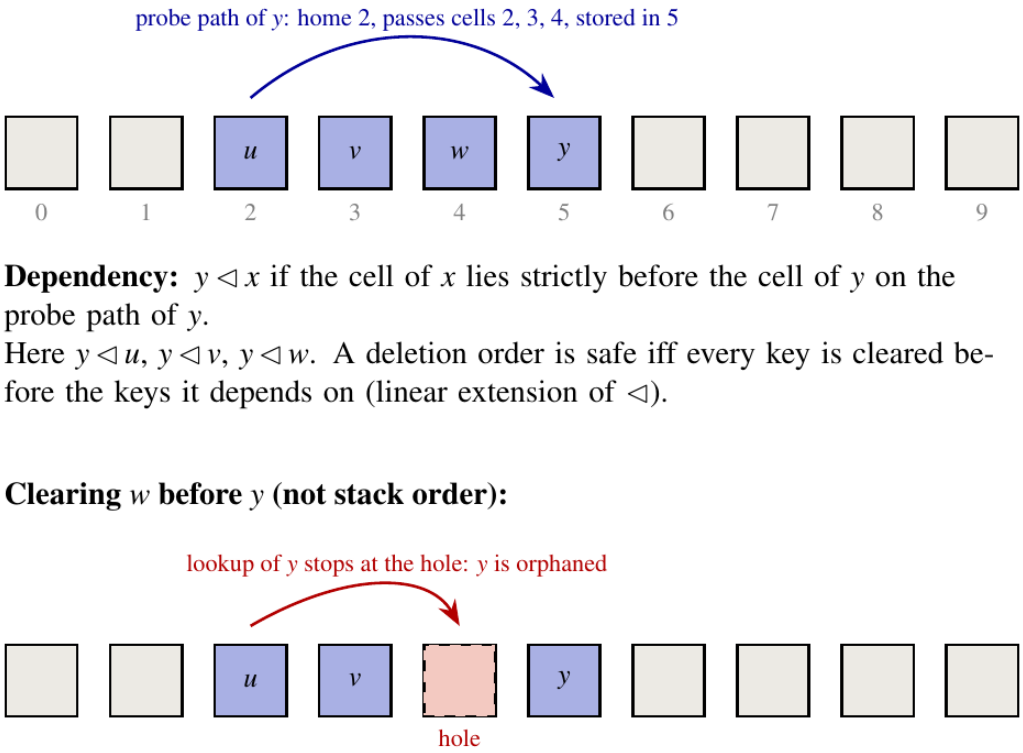


Figure 4. Dependency and orphans. Key *y* probed cells 2, 3 and 4 before it was stored in cell 5, so it depends on *u*, *v* and *w*. Clearing *w* first leaves a hole on its path and orphans *y*; clearing the newest key never does.

Theorem 4. For a given table, the safe orders of clearing cells are exactly the linear extensions of ◁, and stack order is the only order safe for every hash assignment. The number of keys that depend on a key is a queue length minus one; its mean and the limiting fraction of keys with no dependents at load $\alpha = n/m$ are

$$\mathbb{E}[\text{orph}] = \frac{\alpha}{2(1-\alpha)}, \mathbb{P}(\text{orph} = 0) \to \frac{(1-\alpha)(e^{\alpha}-1)}{\alpha} \tag{5}$$

At load 3/4 the mean is 1.5 and only 37% of keys could be removed out of order without harm, so a table that did not follow stack order would need tombstones. The limit law is Janson's [32] and the single-deletion test Sanders' [33]; the characterisation of safe orders, the finite-size law and the reading as orphans are new as stated.

3.3.3 The exact refusal law and the probe limit

In bounded mode the index is static and a declaration that finds no empty slot within L probes is refused. Let $n = m - k$ keys occupy the ring with independent uniform homes (the model for a keyed hash). **Theorem 7.** The probability that a new key finds its first L cells occupied is

$$R_m(n,L) = \frac{k}{m} \sum_{g=L+1}^{n+1} (g-L)\,\pi_g, \pi_g = \frac{B(g)\,Q_{k-1}(m-g)}{Q_k(m)} \tag{6}$$

$$B(g) = \frac{g^{g-1}}{g!}, Q_j(x) = \frac{j\,x^{x-j-1}}{(x-j)!} \tag{7}$$

The ring is cut at its k empty cells into blocks whose sizes are Borel distributed; a block of g cells contains $(g-L)^+$ homes that would be refused. The tail is of order $\rho^L\, L^{-3/2}$ with $\rho = \alpha\, e^{1-\alpha}$, so the Chernoff-type bound previously used to choose L is loose by a factor growing like $L^{3/2}$. The law is classical as a quantity (Knuth [8]; Janson and Viola [9]); the elementary derivation, the finite-size sum, its validation on the real table and its use for design are the contribution. Because of Theorem 2, the same law holds after any number of push/pop pairs on a fixed set of persistent names, and Adhyan Jain's manuscript extends it to a whole fill, an unlimited run and bounded churn above a persistent trunk.

3.3.4 Keyed hash and hash flooding

With the default hash, identifiers can be crafted to collide for every seed, which makes each declaration walk a long probe sequence. STRATA offers a keyed HalfSipHash-1-3 and the probe limit. The law above assumes independent homes, which a key provides only if it is independent of everything the adversary has seen: with a fresh key per table the law holds against arbitrary names, while one persistent key combined with a probe-length oracle lets an adversary plant names that make the limit refuse honest ones (Propositions 13–15 of the theory manuscript). The design rule is therefore a new key per translation unit, derived from a device secret and a counter.

3.4 Modules and their functions

Table 1. Modules of the work, what each does, where it lives in the repository and how it is checked.

Module	Function	Where	Checked by
Table core	declare, lookup, enter, exit in one buffer; all-or-nothing refusal	include/strata/strata.hpp (C++14, header only)	unit, differential and exhaustive tests
Bounded mode	static index, probe limit L, keyed hash	strata.hpp	fuzzing; QEMU bounded runs
Trace recorder	logs every symbol-table call of a real compiler	tools/tcc-trace, tools/chibicc	identical assembly output
Replay harness	replays traces on six tables; heap, allocator calls, time, smallest buffer	benchmarks/	reference model on every answer
Exact undo and orders	Theorem 2, Theorem 4, orphan law	theory/verify	enumeration in exact rationals
Refusal law and design	$R_m(n, L)$; smallest L for a target ε	theory/verify	enumeration; real table (z-scores)
Space ledger	exact word accounting of five layouts	theory/verify/space_model.py	reproduces 24 measured buffers
Interactive tool	Analyse your code, Live lab, results and theory pages	frontend/	npm test (37 checks)

3.5 Flow of the operations

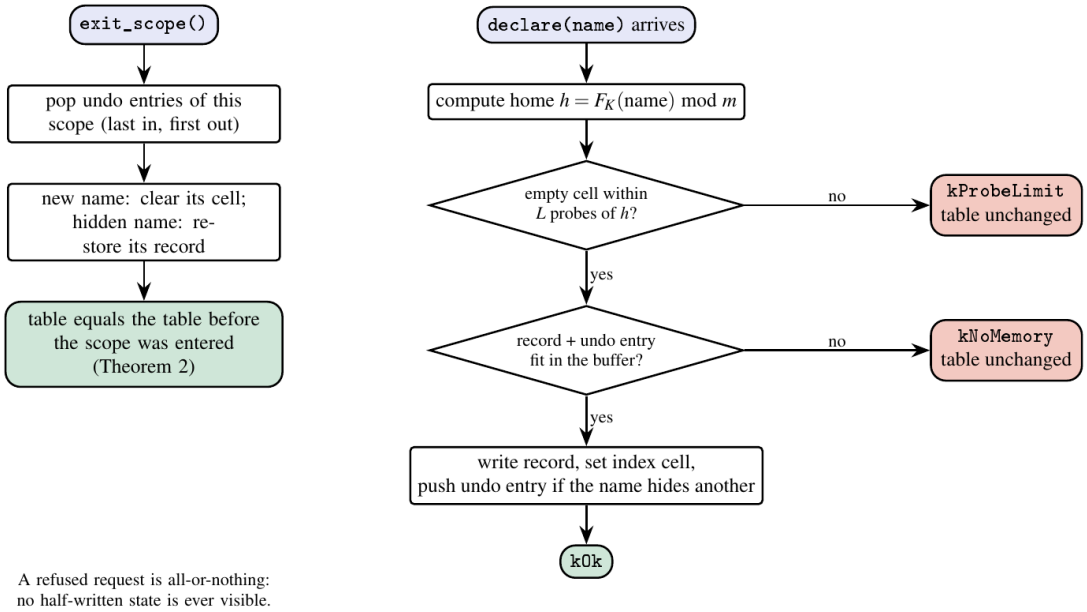


Figure 5. Flow of a declaration and of a scope exit. A declaration is refused, with the table unchanged, when no empty cell is found within L probes or when the record and its undo entry do not fit; a scope exit pops the undo entries in reverse order.

Figure 5 gives the flow. A declaration computes the home slot and probes at most L slots. If no empty slot is found it answers `kProbeLimit`; if the record and the undo entry do not fit it answers `kNoMemory`; in both cases nothing has been written. Otherwise it writes the record, sets the index slot and, if the name hides an outer one, pushes an undo entry. Leaving a scope pops the entries of that scope newest first: a new name has its slot cleared, a hidden name has its binding restored, and the record stack is reset to the scope's mark. By Theorem 2 the table then equals the table that existed before the scope was entered.

The two refusal exits are exactly what the analysis quantifies: `kProbeLimit` is the event of the refusal law, and `kNoMemory` is governed by the space ledger and the measured smallest buffers. Both are all-or-nothing, which is what makes the exact undo meaningful: a refused call never leaves a half-written state.

3.6 Algorithms

Algorithm 1. Declaration and scope exit (all-or-nothing)

```
1:  procedure DECLARE(x, v)
    2:  k ← PackAndHash(x); (r, e) ← Find(k, L)      ▷ record, or first empty slot within L probes
    3:  if r exists and depth(r) = current depth then overwrite value; return kRedeclared
    4:  if r exists then  if fewer than 3 free words then return kNoMemory
        5:  push (r, value, type, depth) on the undo stack; overwrite the binding; return kOk
    6:  if fewer than 3 + w(x) free words then return kNoMemory      ▷ nothing written
    7:  if e = nil then return kProbeLimit      ▷ nothing written
    8:  append the record; store fingerprint and reference in slot e; push e on the undo stack; return
        kOk
9:  procedure EXIT_SCOPE()
    10: while the top undo entry is not the scope's mark do
        11: if it names a slot then clear the slot else restore the saved binding
    12: reset the record stack to the mark; pop the mark
```

Algorithm 1 has the form search, reserve, write: nothing is written before the last point at which the call can be refused. EXIT_SCOPE clears exactly the slots DECLARE wrote, newest first, so Theorem 2 applies and no tombstone is needed.

Algorithm 2. Smallest fixed buffer of a trace (64-byte resolution)

```
1:  input trace T, table variant; output the smallest buffer with which T replays without a refusal
    2:  lo ← 128; hi ← 64 MiB      ▷ hi must pass, else report "does not fit"
    3:  while hi − lo > 64 do
        4:  mid ← ⌊(lo + hi)/2⌋ rounded down to a multiple of 64
        5:  if Replay(T, mid) refuses nothing then hi ← mid else lo ← mid
    6:  return hi      ▷ the same search runs for STRATA and the conventional table
```

Algorithm 2 is benchmarks/budget_main.cpp. Because sizing is not perfectly monotone, the harness re-checks the result at 1×, 1.25×, 2× and 4× the buffer and reports any refusal.

Algorithm 3. Design rule: smallest probe limit for a target failure probability

```
1:  input m, n, target ε; output the smallest L with R_m(n, L) ≤ ε
    2:  lo ← 2; hi ← m
    3:  while lo < hi do
        4:  mid ← ⌊(lo + hi)/2⌋
        5:  if R_m(n, mid) ≤ ε then hi ← mid else lo ← mid + 1
    6:  return lo
```

Algorithm 3 turns the exact law (Equation 6) into a design value; R_m decreases in L, so bisection applies.

Contribution of the team. (C1) The STRATA table: one buffer, all-or-nothing refusal, exact scope exit, bounded mode. (C2) A measurement method on recorded compiler traces with a reference model, a fair allocator-free baseline and paired, interleaved timing. (C3) Exact undo, safe deletion orders and the exact refusal law used as design rules, with a space ledger that explains the measured buffer saving. (C4) An interactive tool that runs any pasted C program's symbol-table calls on the real table.

4. Implementation and result analysis

4.1 Tools, configuration and commands

STRATA is a header-only C++14 library (`include/strata/strata.hpp`, 676 lines) compiled with g++ 11.4.0 at `-O2` [34]. Workloads were recorded by adding logging to the Tiny C Compiler [35] at the points where it looks up an identifier, pushes a symbol and opens or closes a scope, and compiling eight open-source C projects at pinned versions; a second set of traces comes from an instrumented chibicc [36]. The harness replays each trace on STRATA, the conventional allocator-free single-buffer table, an undo-log table on std::unordered_map (the algorithm of LLVM's ScopedHashTable [7]), the same on a Swiss-table map [37], one map per scope, and an interned Swiss flat map. Timing used 31 paired, interleaved rounds per trace on an Intel Core Ultra 7 258V under WSL 2, with 95% bootstrap intervals; heap peak and allocator calls are counted by a replacement allocator.

The bare-metal run cross-compiles the same header with arm-none-eabi-g++ for the mps2-an385 board (Cortex-M3) and runs it under QEMU [38, 39] with RAM limited to 64 KiB, no heap and no operating system. The commands are `bash scripts/test.sh` (unit and differential tests), `bash scripts/run_all.sh <label>` (all measurements), `python scripts/analyze.py` and `manuscript/tools/make_numbers.py` (every number of this article) and `bash theory/verify/run_all.sh` (every check of the theory). The interactive tool is a React [40] and Vite [41] application whose local server runs a C++ engine linked with the real table (`cd frontend/web; npm start`); Figure 6 shows its Analyse page.

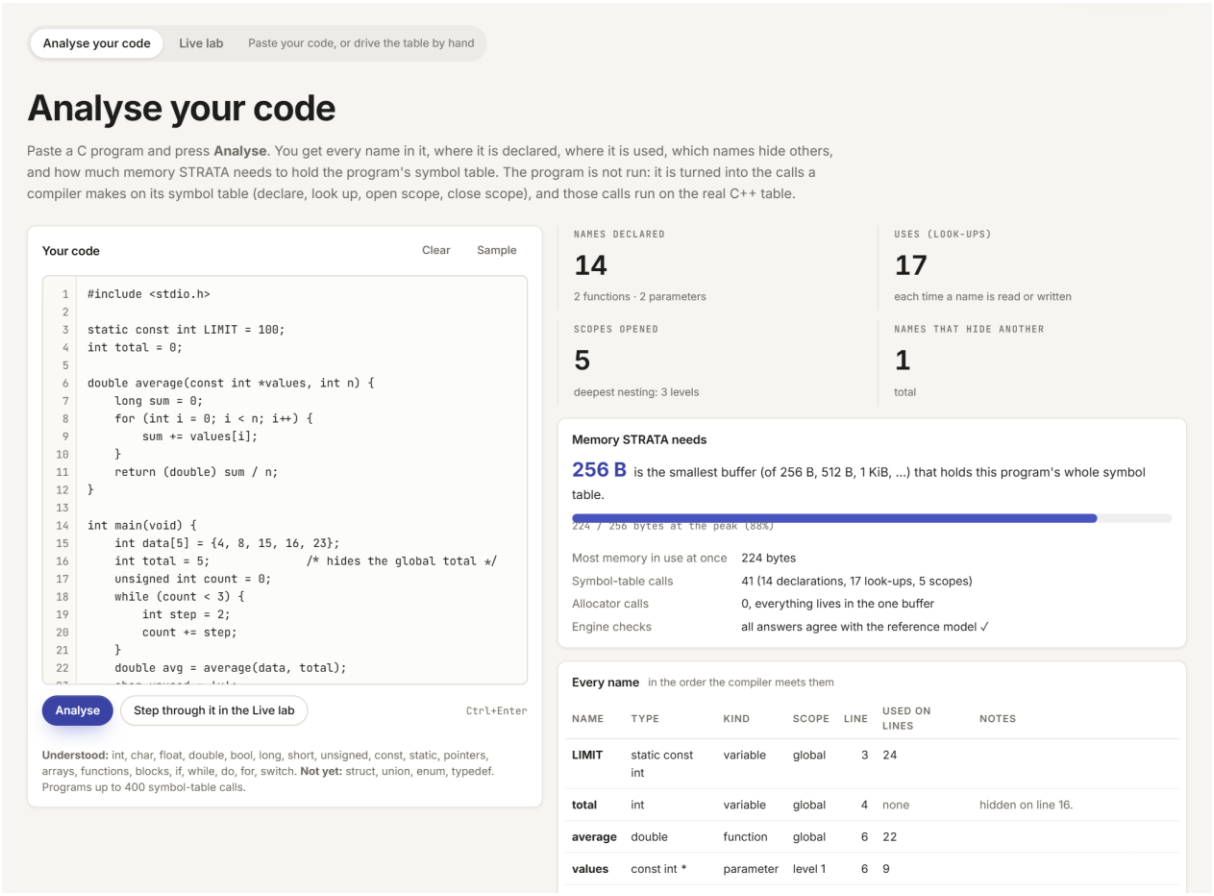


Figure 6. The implementation tool: “Analyse your code”. A pasted C program is translated into the calls a compiler makes on its symbol table, which run on the real strata::Table in the local engine; the page lists every name with its type, scope, line and uses, the names it hides, problems such as unused names, and the smallest buffer that holds the program.

4.2 Parameter setup

Table 2. Parameters of the experiments.

Parameter	Values	Used for
Compiler traces	8 projects (cJSON, littlefs, zlib, Lua, FreeRTOS, Mbed TLS, TinyCC, SQLite), 15 traces in all with chibicc	all metrics
Tables compared	STRATA, conventional single-buffer table, std map + undo log, Swiss map + undo log, map per scope, interned Swiss map	memory, time
Timing	31 paired interleaved rounds; median ratio, 95% bootstrap interval	replay time
Buffer search	128 B to 64 MiB, 64-byte resolution (Algorithm 2)	smallest buffer
Index load (bounded mode)	at most 3/4; probe limit L = 512 in the library documentation	probe limit
Exact law	$m = 2^8 \ldots 2^{16}$ and $m \to \infty$; $\varepsilon = 10^{-6}, 10^{-9}$; load 3/8 … 3/4	design table
Real-table trials	400 000 per setting, 7 settings, keyed hash, fresh key per trial	law validation
Hash flooding	1 024 to 32 768 names of 150 characters, median of 5 runs	robustness
Embedded target	Cortex-M3 (mps2-an385) in QEMU, 64 KiB RAM	fit, code size
Machine	Intel Core Ultra 7 258V, g++ 11.4.0, WSL 2	all timings

4.3 Performance analysis

Eight metrics are reported, each with a table or figure and the reason for the result. The workloads are summarised in Table 4.

Table 3. The eight compiler traces. “Names” counts distinct identifiers; “Peak live” is the largest number of simultaneously live declarations.

Project	Units	Events	Declares	Lookups	Names	Peak live	Depth
cJSON	2	20 874	5 247	9 194	1 366	1 067	10
littlefs	2	21 080	3 835	12 230	1 001	732	16
zlib	15	58 058	16 341	27 405	1 041	571	23
Lua	34	188 509	58 619	86 887	3 622	1 365	14
FreeRTOS	9	37 875	10 346	17 417	2 226	1 505	16
Mbed TLS	105	983 008	319 886	438 876	7 622	2 302	14
TinyCC	1	89 135	13 776	54 623	6 704	5 509	27
SQLite	1	268 960	27 072	172 426	7 801	4 245	22

Metric 1: correctness

Table 4. Correctness evidence. Every answer of every replay is compared with an independent reference model of scoped binding.

Check	Size	Result
Answers on the eight compiler traces, against the reference model	1 274 180 lookups and declarations	0 wrong
Unit and differential tests	2 732 358 checks	0 failed
Exhaustive enumeration of short operation sequences	80 621 568 sequences, 725 594 112 operations	0 failures
Refused calls leave the state unchanged (digest compared)	130 361 632 refusals in the enumeration	all unchanged
chibicc with STRATA inside, assembly compared with the original	158 of 158 compiled units	byte-identical
Bare-metal runs, checksum against the desktop run	5 default + 5 bounded	all pass

STRATA gave no wrong answer and no check failed. The enumeration matters most for requirement R2: it covers every short sequence of calls on small buffers, including every way a call can be refused, and compares the table's digest before and after each refusal.

Metric 2: smallest fixed buffer

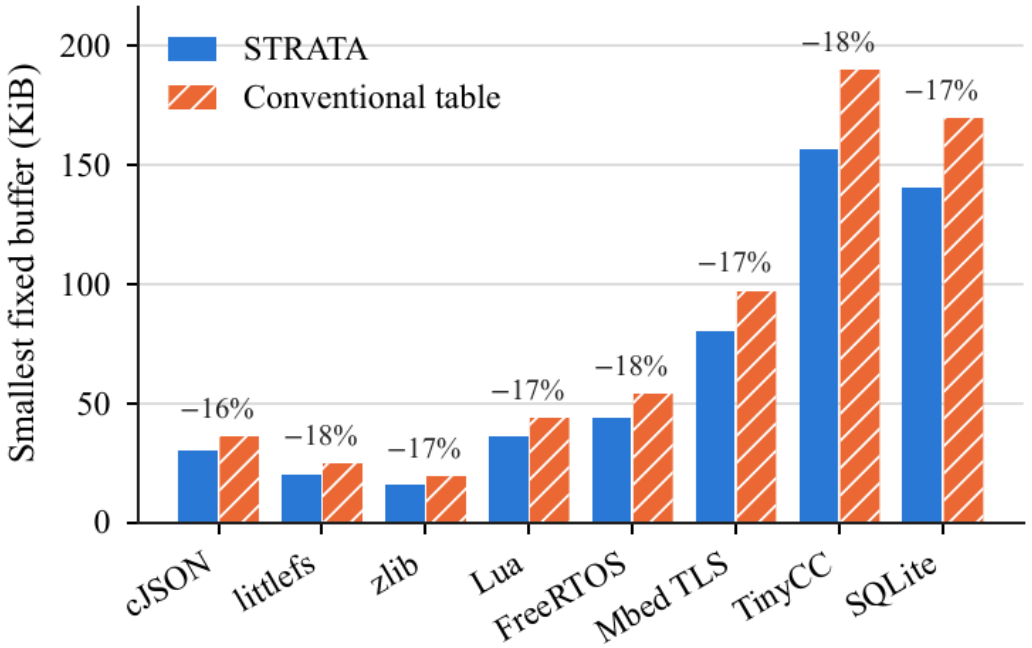


Figure 7. Smallest fixed buffer that replays each trace without a refusal, for STRATA and the conventional single-buffer table (Algorithm 2).

Table 5. Smallest fixed buffer on the eight compiler traces.

Project	STRATA (KiB)	Conventional (KiB)	Saving (%)	Bytes per live decl.
cJSON	30.0	35.9	16.4	28.8
littlefs	20.0	24.2	17.5	28.0
zlib	15.8	18.9	16.6	28.2
Lua	35.9	43.4	17.3	27.0
FreeRTOS	44.0	53.6	17.9	29.9
Mbed TLS	80.0	96.9	17.4	35.6
TinyCC	156.5	189.8	17.5	29.1
SQLite	140.4	169.4	17.2	33.9

STRATA needs a buffer 16.4–17.9% smaller than the conventional allocator-free table on every trace, from 15.8 KiB (zlib) to 156.5 KiB (TinyCC). The saving is nearly constant because it is a per-declaration difference in layout; Metric 4 decomposes it exactly.

Metric 3: heap peak and allocator calls

Table 6. Heap peak in growable mode (KiB).

Project	STRATA	Conventional	METIS-X	Map per scope	Undo log	Interned map
cJSON	42.9	42.8	102.7	72.0	116.0	140.4
littlefs	25.8	25.7	106.9	54.1	67.8	141.9
zlib	17.2	25.7	52.6	44.4	60.3	71.1
Lua	64.2	96.2	106.8	96.7	127.7	142.3
FreeRTOS	57.8	57.7	217.3	116.1	147.3	293.1
Mbed TLS	144.3	144.2	234.4	189.4	277.7	310.9
TinyCC	324.5	324.4	423.0	424.5	550.8	575.6
SQLite	194.8	194.7	451.0	333.3	513.5	1125.6

Against tables that allocate node by node the difference is large: STRATA's heap peak is 23–76% (median 58%) below that of METIS-X, its multi-allocation predecessor, and it makes 98.7–99.9% fewer allocator calls. The conventional single-buffer table reaches the same heap peak as STRATA on 6 of 8 traces in growable mode, because both grow one block in steps; most of the gain over allocator-based tables therefore comes from keeping everything in one allocation-free buffer, not from the compact layout.

Metric 4: where the buffer saving comes from (space ledger)

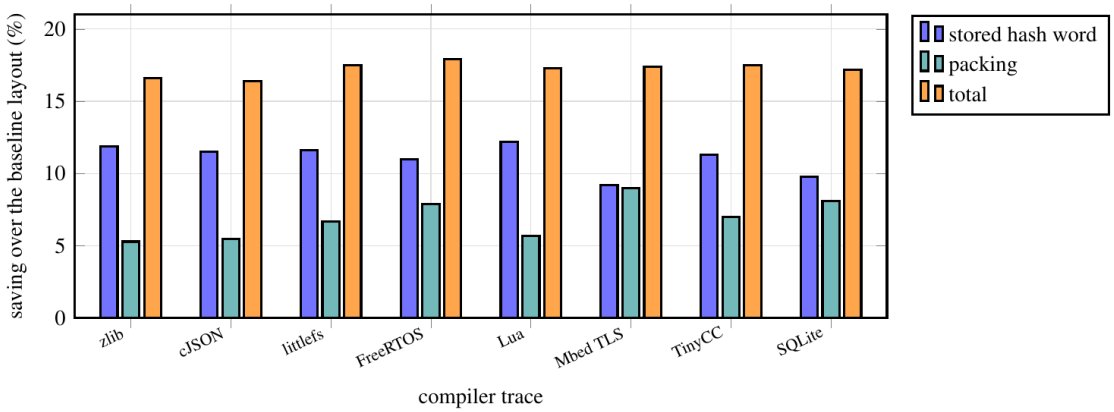


Figure 8. Saving of the compact layout over the conventional single-buffer layout on the eight traces, split into the stored hash word and name packing (space ledger).

Table 7. Smallest buffer for the conventional layout (C_0), the same without a stored hash word (C_1) and STRATA (S), with the savings of dropping the hash, of packing names, and in total.

trace	C_0 (KiB)	C_1	S	hash %	pack %	total %
zlib	18.9	16.6	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	140.4	9.8	8.1	17.2

The exact word accounting reproduces all 24 measured minimum buffers to the 64-byte resolution of the measurement. Of the 16.4–17.9% saving, 9.2–12.2% comes from not storing a hash word and 5.3–9.0% from packing names; interning with the undo log changes the footprint by between −0.38% and 0.00%. Against a conventional table that does not store the hash, STRATA would be only 5–9% smaller. This corrects the earlier impression that interning was the source of the compactness.

Metric 5: replay time

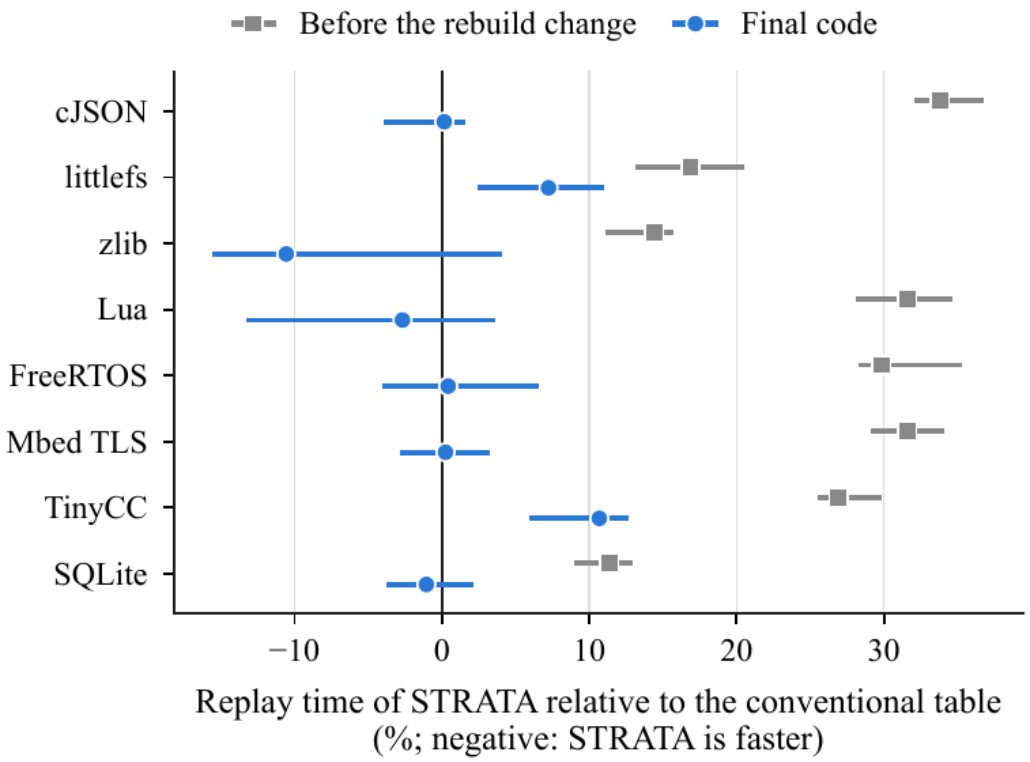


Figure 9. Replay time of STRATA relative to the conventional single-buffer table: median paired difference with 95% interval over 31 rounds (final code), and the same before an index-rebuild defect was fixed.

Table 8. Replay time: median ratio in percent with 95% bootstrap interval; negative means STRATA is faster; "n.e." means not established.

Project	STRATA vs conventional (%)		STRATA vs METIS-X (%)	
cJSON	+0.2 [−3.9, +1.6]	n.e.	−22.8 [−27.5, −19.1]	faster
littlefs	+7.2 [+2.4, +11.0]	slower	−24.5 [−27.2, −18.1]	faster
zlib	−10.6 [−15.6, +4.1]	n.e.	−29.9 [−32.8, −27.4]	faster
Lua	−2.7 [−13.3, +3.6]	n.e.	−30.9 [−35.3, −29.2]	faster
FreeRTOS	+0.4 [−4.0, +6.6]	n.e.	−33.9 [−39.0, −31.3]	faster
Mbed TLS	+0.2 [−2.8, +3.3]	n.e.	−36.7 [−39.6, −32.4]	faster
TinyCC	+10.7 [+5.9, +12.7]	slower	−20.4 [−25.7, −18.7]	faster
SQLite	−1.1 [−3.7, +2.1]	n.e.	−15.9 [−20.7, −6.3]	faster

Against the conventional table STRATA is faster on 0 traces, slower on 2 and not established on 6; where a difference is established it is at most 10.7%. Against METIS-X it is faster on 8 of 8 traces, by 15.9–36.7%. The compact table is therefore not faster than a well-built allocator-free table; its contribution is memory that is fixed, smaller and refusable, at comparable speed. The grey series shows why the method matters: an earlier version rebuilt the index on every buffer growth and appeared 11–34% slower, an artefact the paired design exposed.

Metric 6: accuracy and tightness of the exact refusal law

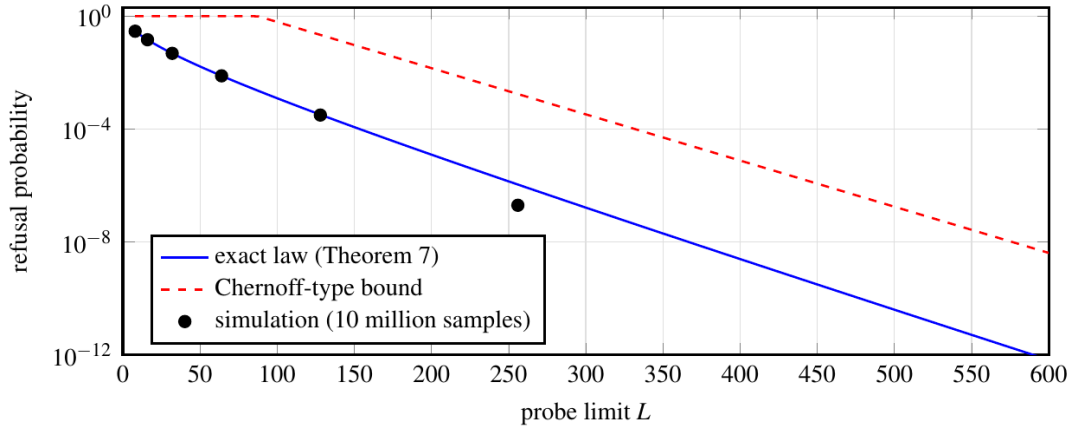


Figure 10. Probability that an insertion needs more than L probes (m = 65 536, load 3/4): exact law, Chernoff-type bound and the stored simulation (10 million samples from 500 tables).

Table 9. Exact law against the real probe-limited table in 400 000 trials per setting.

m	n	L	observed	exact	z	clean trials	mismatches
256	128	8	0.03311	0.03315	-0.13	254 871	0
256	160	16	0.02643	0.02667	-0.94	307 410	0
256	190	24	0.05740	0.05730	+0.26	247 303	0
256	190	40	0.01372	0.01375	-0.19	369 939	0
1024	760	48	0.01251	0.01286	-1.96	307 286	0
1024	760	96	0.00060	0.00058	+0.52	397 066	0
1024	512	24	0.00040	0.00046	-1.63	395 607	0

Table 10. Smallest probe limit with per-insertion refusal probability at most ε: exact law against the Chernoff-type rule.

load	ε	$m=2^8$	2^{10}	2^{12}	2^{14}	2^{16}	$m\to\infty$	Chernoff
3/8	10^{-6}	25	28	29	29	29	29	43
3/8	10^{-9}	37	44	46	47	47	47	62
1/2	10^{-6}	42	49	51	52	52	52	81
1/2	10^{-9}	59	77	83	84	85	85	117
5/8	10^{-6}	70	93	101	104	104	104	171
5/8	10^{-9}	95	142	162	168	170	170	244
3/4	10^{-6}	121	201	242	255	258	260	455
3/4	10^{-9}	148	289	380	413	422	426	638

All seven z-scores lie within ±1.96 and the lock-step comparison found no mismatch, so the law describes the implementation, not only the model. At load 3/4 and $\varepsilon = 10^{-6}$ the exact limit is 258 for 2^{16} slots against 455 for the Chernoff-type rule, 43% fewer probes allowed per call; the library's documented limit of 512 corresponds to a refusal probability of about 2.4×10^{-11} per declaration at that size and is safe but conservative. A smaller limit bounds the work on hostile input more tightly at a stated failure probability.

Metric 7: robustness under hash flooding

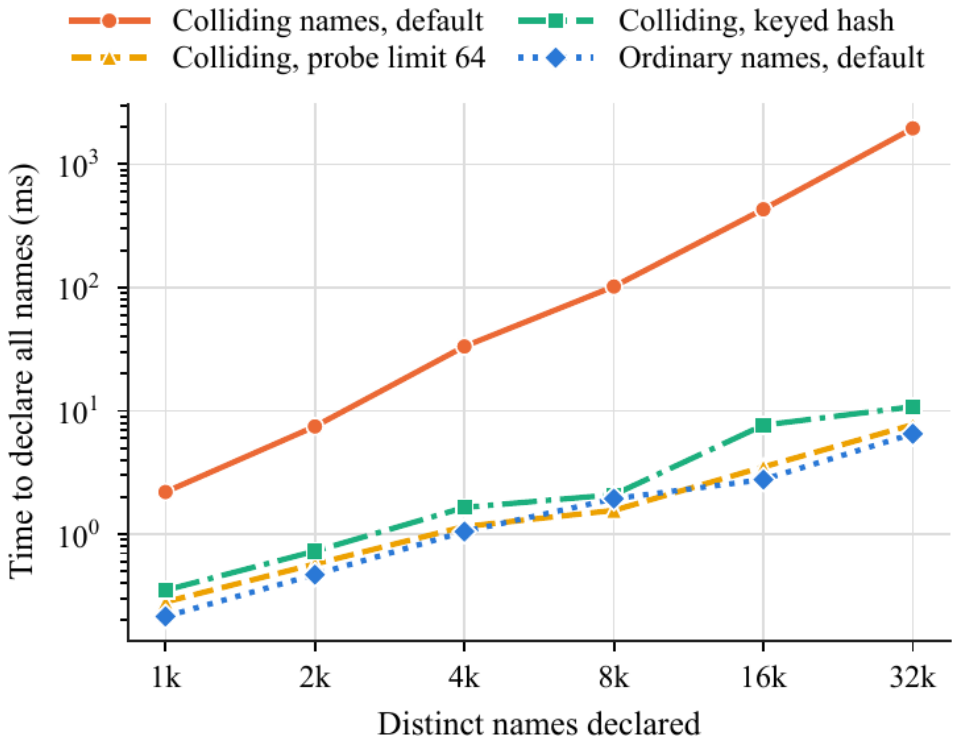


Figure 11. Time to declare all names: ordinary names and names crafted to collide in the default hash, with the default hash, the keyed hash and a probe limit (median of 5 runs, log scale).

Table 11. Declaration time with the default hash: ordinary against colliding identifiers.

Names	Ordinary (ms)	Colliding (ms)	Slowdown (×)	Colliding lookups (ms)
1 024	0.22	2.20	10	1.74
2 048	0.47	7.47	16	6.85
4 096	1.05	33.28	32	26.95
8 192	1.94	101.91	53	88.86
16 384	2.77	430.92	155	381.88
32 768	6.52	1953.86	300	1684.09

Identifiers that collide for every seed slow declarations by a factor of 300 at 32 768 names (1954 ms against 6.5 ms), growing roughly quadratically. The keyed hash brings colliding names back to 1.2× the ordinary time, at a cost of 20.2–32.2% replay time on ordinary traces, and the probe limit alone refuses colliding names instead of slowing down. Requirement R3 therefore needs the bounded mode.

Metric 8: fit on a 64 KiB microcontroller

Table 12. Cortex-M3 (mps2-an385) under QEMU, 64 KiB RAM, no heap, no operating system. Emulated, not real hardware.

Quantity	Value
Bare-metal runs passed (default build)	5 of 5, checksum equal to the desktop run
Bare-metal runs passed (bounded build)	5 of 5
STRATA code size	2 715 bytes
Stack used	492 bytes (default build); 484 bytes (bounded build)
Workloads that fit only with STRATA at the same buffer	3 (FreeRTOS at 48 KiB, Lua at 40 KiB, cJSON at 32 KiB)

The same header runs unchanged without a heap or an operating system. At the same buffer, STRATA fits three workloads that the conventional table refuses (for example FreeRTOS in 48 KiB, where the conventional table refuses 420 operations), which is the practical meaning of a 16–18% smaller buffer on a device whose RAM is fixed.

4.4 Comparison with related work

Three works are used for comparison. **(1) LLVM's ScopedHashTable** [7], measured here as the undo-log table on std::unordered_map: STRATA's heap peak is 61% lower in median (41–71%), it makes 99.2–99.9% fewer allocator calls, and it is 7–34% faster on all eight traces, while offering a fixed buffer and defined refusal, which the map design does not. **(2) Optimal open addressing without reordering** [19]: their bounds cover insertions only; Theorem 2 transfers them to every push of a stack-disciplined workload. **(3) Bounded linear probing** [21] analyses a process in which a key that exceeds the probe cap is discarded and redrawn; our event, refuse and stop, with stack-order deletions, is different, and the exact law gives its probability at finite size.

Table 13. Comparison with the related work used for evaluation.

Aspect	LLVM ScopedHashTable design	Insertion-only bounds (FKK)	Bounded linear probing (ABB)	STRATA (this work)
Memory	allocator, node per declaration	—	—	one caller buffer, no allocator
At the limit	undefined (allocation failure)	—	key discarded and redrawn	refused, state unchanged
Deletions	undo log	not treated	not treated	exact undo; stack order only safe order
Probe limit	none	not a design rule	asymptotic results	exact law; 258 at load 3/4, 10^{-6}
Evidence	production use	proofs	proofs	8 compiler traces, proofs, real-table validation

5. Conclusion and scope for future work

- **The proposed work.** STRATA is a scoped symbol table for compilers under a fixed memory budget: one caller-supplied buffer, no allocator calls, refusal that changes nothing, and an optional bound on the work of every call.
- **Existing methodology.** Textbook and production tables (an undo log on a hash map, one map per scope) depend on the allocator, and the analysis of open addressing covers insertions, so probe limits were set with loose Chernoff-type bounds.
- **Proposed methodology and its advantage.** Keeping index, records and undo stack in one buffer makes memory fixed and measurable; exact undo explains why scope exit needs no tombstones, and the exact refusal law turns the probe limit into a design value.

- **Tools.** A recording Tiny C Compiler and chibicc, a C++14 replay harness with a reference model, QEMU for the Cortex-M3, exact-arithmetic verification programs and a React tool linked to the real table.
- **Results.** No wrong answer in 1 274 180 checked operations; a fixed buffer 16.4–17.9% smaller than a conventional single-buffer table, explained exactly as a stored hash word plus name packing; speed not distinguishable on six of eight traces and at most 10.7% slower where it differs; 98.7–99.9% fewer allocator calls than the predecessor; a probe limit of 258 instead of 455 at load 3/4 for 10^{-6}; three workloads that fit a 64 KiB microcontroller only with STRATA.
- **Scope for future work.** A limit law for refusals on wide and deep scope trees is open. The measurements cover C only, one machine and an emulated microcontroller; C++ workloads, real hardware and a compiler that keeps several scopes alive at once are next. A machine-checked version of the proofs and a measured timing channel for the key-lifetime results would strengthen the claims, and the authors still have to review every proof in full.

6. References

[1] Gurdeep Singh, Robbert and Christophe Scholliers. (2019) “WARDuino: a dynamic WebAssembly virtual machine for programming microcontrollers.” Proceedings of the 16th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR 2019): 27–36. doi:10.1145/3357390.3361029.

[2] Rouvoet, Arjen, Hendrik van Antwerpen, Casper Bach Poulsen, Robbert Krebbers, and Eelco Visser. (2020) “Knowing when to ask: sound scheduling of name resolution in type checkers derived from declarative specifications.” Proceedings of the ACM on Programming Languages (OOPSLA) 4 (OOPSLA): 1–28 (Crossref gives no article number). doi:10.1145/3428248.

[3] van Antwerpen, Hendrik and Eelco Visser. (2021) “Scope States: Guarding Safety of Name Resolution in Parallel Type Checkers.” 35th European Conference on Object-Oriented Programming (ECOOP 2021), LIPIcs vol. 194 194 (1 (article number)): 1:1–1:29. doi:10.4230/LIPIcs.ECOOP.2021.1.

[4] Csikor, Levente, Dinil Mon Divakaran, Min Suk Kang, et al. (2019) “Tuple Space Explosion: A Denial-of-Service Attack against a Software Packet Classifier.” Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '19): 292–304. doi:10.1145/3359989.3365431.

[5] Appel, Andrew W. (1998) “Modern Compiler Implementation in C.” Cambridge University Press. doi:10.1017/CBO9781139174930.

[6] Graham, S. L., W. N. Joy, and O. Roubine. (1979) “Hashed Symbol Tables for Languages With Explicit Scope Control.” Proc. SIGPLAN Symposium on Compiler Construction: 50–57. doi:10.1145/800229.806953.

[7] Project, LLVM. (n.d.) “ScopedHashTable.h, LLVM ADT library.”

[8] Knuth, Donald E. (1998) “The Art of Computer Programming, Volume 3: Sorting and Searching.” Addison-Wesley.

[9] Janson, Svante and Alfredo Viola. (2016) “A Unified Approach to Linear Probing Hashing with Buckets.” Algorithmica 75 (4): 724–781. doi:10.1007/s00453-015-0111-x.

[10] Bender, Michael A., Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. (2023) “Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once.” Journal of the ACM 70 (6): 1–51. doi:10.1145/3625817.

[11] Bender, Michael A., Martin Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. (2022) “On the optimal time/space tradeoff for hash tables.” Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC '22): 1284–1297. doi:10.1145/3519935.3519969.

[12] Dalal, Ketan, Luc Devroye, and Ebrahim Malalla. (2023) “Two-Way Linear Probing Revisited.” Algorithms 16 (11): 500. doi:10.3390/a16110500.

[13] Hu, Yang, William Kuszmaul, Jingxun Liang, Stefan Walzer, Huacheng Yu, and Renfei Zhou. (2026) “Quadratic Probing Revisited: Smoothed Analysis and the Fall of Robin Hood.” arXiv preprint (cs.DS): arXiv:2607.13247. https://arxiv.org/abs/2607.13247.

[14] He, Le and Hongbo Yu. (2023) “Cryptanalysis of Reduced-Round SipHash.” The Computer Journal 67 (3): 875–883. doi:10.1093/comjnl/bxad026.

[15] Kenyon, Richard and Mei Yin. (2023) “Parking Functions: From Combinatorics to Probability.” Methodology and Computing in Applied Probability 25 (1): Article 32. doi:10.1007/s11009-023-10022-5.

[16] Singh, Shubhi, and Adhyan Jain. (2026) “STRATA: a scoped symbol table in one caller-supplied buffer.” Public repository cdesproject (https://github.com/shubhisingh1510/cdesproject); paper/, theory/ and frontend/ hold the sources of this article, its proofs, its measurements and the interactive tool.

[17] Bender, Michael A., Bradley C. Kuszmaul, and William Kuszmaul. (2021) “Linear Probing Revisited: Tombstones Mark the Demise of Primary Clustering.” 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS): 1171–1182. doi:10.1109/FOCS52979.2021.00115.

[18] Braverman, Mark and William Kuszmaul. (2024) “Tight Analyses of Ordered and Unordered Linear Probing.” 2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS): 606–635. doi:10.1109/FOCS61266.2024.00046.

[19] Farach-Colton, Martin, Andrew Krapivin, and William Kuszmaul. (2024) “Optimal Bounds for Open Addressing Without Reordering.” 2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS): 594–605. doi:10.1109/FOCS61266.2024.00045.

[20] Krapivin, Andrew, William Kuszmaul, and Yixuan Wang. (2026) “Linear Probing with Non-Greedy Insertions.” arXiv preprint (cs.DS): arXiv:2607.17494. https://arxiv.org/abs/2607.17494.

[21] Alharbi, Ahmed, Cyril Banderier, and Charles Bouillaguet. (2026) “Bounded Linear Probing Hashing.” 37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026), LIPIcs vol. 381 381: 28:1–28:17. doi:10.4230/LIPIcs.AofA.2026.28.

[22] Bender, Michael A., William Kuszmaul, and Renfei Zhou. (2025) “Optimal Non-oblivious Open Addressing.” Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC '25): 268–277. doi:10.1145/3717823.3718215.

[23] Bender, Michael A., Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. (2023) “Tiny Pointers.” Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA): 477–508. doi:10.1137/1.9781611977554.ch21.

[24] Aguilar-Fraga, Tomas, Jennifer Elder, Rebecca E. Garcia, et al. (2024) “Interval and l-interval rational parking functions.” Discrete Mathematics & Theoretical Computer Science 26:1 (Permutation Patterns 2023): #10 (article id 12598). doi:10.46298/dmtcs.12598.

[25] van Antwerpen, Hendrik, Casper Bach Poulsen, Arjen Rouvoet, and Eelco Visser. (2018) “Scopes as types.” Proceedings of the ACM on Programming Languages (OOPSLA) 2 (OOPSLA): 1–30 (Crossref gives no article number). doi:10.1145/3276484.

[26] Zwaan, Aron, Hendrik van Antwerpen, and Eelco Visser. (2022) “Incremental type-checking for free: using scope graphs to derive incremental type-checkers.” Proceedings of the ACM on Programming Languages (OOPSLA2) 6 (OOPSLA2): 424–448. doi:10.1145/3563303.

[27] Ball, Thomas, Peli de Halleux, and Michał Moskal. (2019) “Static TypeScript: an implementation of a static compiler for the TypeScript language.” Proceedings of the 16th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR 2019): 105–116. doi:10.1145/3357390.3361032.

[28] Mochizuki, Fumika, Tetsuro Yamazaki, and Shigeru Chiba. (2025) “BlueScript: A Disaggregated Virtual Machine for Microcontrollers.” The Art, Science, and Engineering of Programming 10 (3): Article 21. doi:10.22152/programming-journal.org/2025/10/21.

[29] Naor, Moni and Eylon Yogev. (2019) “Bloom Filters in Adversarial Environments.” ACM Transactions on Algorithms 15 (3): 1–30. doi:10.1145/3306193.

[30] Clayton, David, Christopher Patton, and Thomas Shrimpton. (2019) “Probabilistic Data Structures in Adversarial Environments.” Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19): 1317–1334. doi:10.1145/3319535.3354235.

[31] Blair, William, Andrea Mambretti, Sajjad Arshad, et al. (2020) “HotFuzz: Discovering Algorithmic Denial-of-Service Vulnerabilities Through Guided Micro-Fuzzing.” Proceedings 2020 Network and Distributed System Security Symposium (NDSS). doi:10.14722/ndss.2020.24415.

[32] Janson, Svante. (2005) “Individual Displacements for Linear Probing Hashing With Different Insertion Policies.” ACM Transactions on Algorithms 1 (2): 177–213. doi:10.1145/1103963.1103964.

[33] Sanders, Peter. (2018) “Hashing with Linear Probing and Referential Integrity.” arXiv preprint arXiv:1808.04602.

[34] ISO/IEC (International Organization for Standardization / International Electrotechnical Commission). (2014) “ISO/IEC 14882:2014, Information technology - Programming languages - C++ (Edition 4).” ISO/IEC standard, published December 2014. https://www.iso.org/standard/64029.html. Accessed October 2026.

[35] Bellard, Fabrice, and TinyCC contributors. (2026) “Tiny C Compiler, commit 43c7708.” https://repo.or.cz/tinycc.git. Accessed October 2026.

[36] Ueyama, Rui. (2026) “chibicc: a small C compiler, commit 90d1f7f.” https://github.com/rui314/chibicc. Accessed October 2026.

[37] Popovitch, Gregory. (2023) “The Parallel Hashmap, version 1.4.1.” https://github.com/greg7mdp/parallel-hashmap. Accessed October 2026.

[38] Bellard, Fabrice. (2005) “QEMU, a Fast and Portable Dynamic Translator.” 2005 USENIX Annual Technical Conference (USENIX ATC 05), FREENIX Track. https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator. Accessed October 2026.

[39] The QEMU Project Developers. (2026) “Arm MPS2 and MPS3 boards (mps2-an385, mps2-an386, mps2-an500, mps2-an505, mps2-an511, mps2-an521, mps3-an524, mps3-an536, mps3-an547, mps3-an555) - QEMU documentation.” QEMU documentation (docs for QEMU 11.1.50, a development version). https://www.qemu.org/docs/master/system/arm/mps2.html. Accessed October 2026.

[40] Meta Platforms, Inc. and React contributors. (n.d.) “React (react.dev documentation website).” Website. https://react.dev/. Accessed October 2026.

[41] VoidZero Inc. and Vite contributors. (n.d.) “Getting Started | Vite.” Website (Vite documentation). https://vite.dev/guide/. Accessed October 2026.

Author Contributions

Shubhi Singh designed and implemented STRATA, its tests, the trace recorder, the benchmarks and the interactive tool, and wrote the empirical study. Adhyan Jain wrote the earlier METIS-X table that STRATA follows, directed the theoretical work (the adversarial review, the theorems and proofs, the verification programs and the Theory pages) and produced the course-template manuscript this version extends. Both authors are responsible for the content; roles follow docs/CONTRIBUTIONS.md.

Acknowledgements

The authors thank their guide, Prof. Bhuvaneshwari M, School of Computer Science and Engineering, VIT Vellore, and the maintainers of the open-source tools used: GCC, QEMU, Python, TeX, React and Vite.

Funding

This research received no external funding.

Competing interests

The authors declare no competing interests.

Data and code availability

All sources, traces, raw results and programs are in the public repository [16]. `manuscript/tools/make_numbers.py` regenerates every measured number, `bash theory/verify/run_all.sh` every theoretical one, and `python manuscript/final/build_final.py` this document.